

SIMPLE TEXT EDITOR:

LOGIC

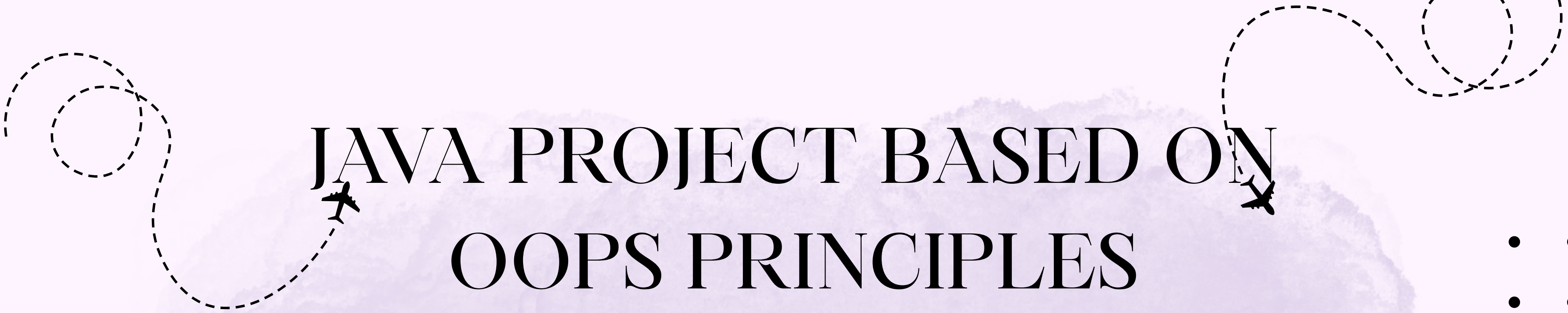
PRESENTATION

JAVA PROJECT

Submitted to: Mr. Santanu Sasmal
IBM SME

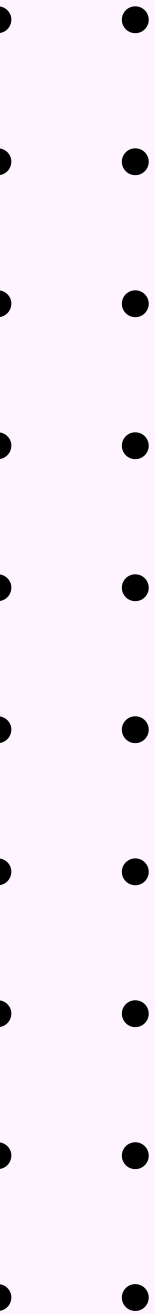
by Apurva Agarwal

ERP: RU-25-10265
IBM 2A



JAVA PROJECT BASED ON OOPS PRINCIPLES

- THE 4 OOPS PRINCIPLES ARE:
- DATA ABSTRACTION
- ENCAPSULATION
- INHERITANCE
- POLYMORPHISM





INTRODUCTION

A text editor is a software application used to create, edit, and manage textual content. Text editors are widely used in programming, documentation, note taking, and content creation.

This project focuses on developing the core logic of a simple text editor using Java and Object-Oriented Programming (OOP) principles.

The project demonstrates:

- Classes and Objects
- Encapsulation
- StringBuffer (Mutable Strings)
- Method Design
- File Handling

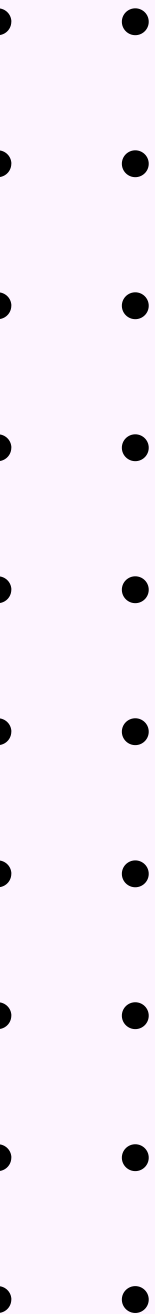
- •
- •
- •
- •
- •
- •
- •
- •
- •





OBJECTIVE OF THE PROJECT

- To understand Object-Oriented Programming concepts.
- To implement encapsulation and data hiding.
- To learn the use of StringBuffer for mutable string operations.
- To design methods for text manipulation.
- To build a simple real-world application using Java.



WORKING OF THE PROJECT

- The project consists of a Document Class that stores text using a private StringBuffer object.

Step 1: Create Class

- Represents the text editor.
- Manages all text operations.

Step 2: Initialize StringBuffer

- Stores text internally.
- Prevents direct access from outside the class.

Step 3: Constructor

- Initializes an empty text buffer.

Step 4: Add Text

- Appends user-entered text.

Step 5: Clear Text

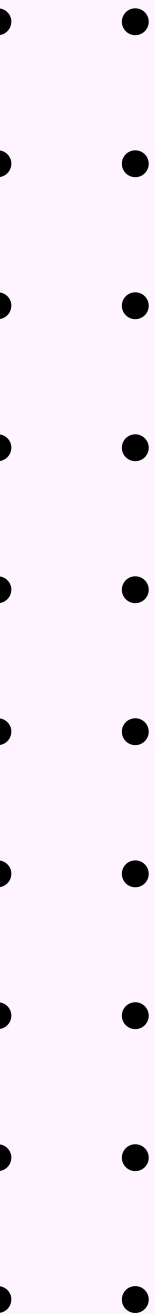
Removes all existing text

Step 6: Reverse Text

- Reverses the current content.

Step 7: Display Text

- Shows the current text stored in the editor.



SCREENSHOTS

```
Document.java | 2 X Document.java C:\...texteditor DocumentDemo.java TextEditorApp.java Te
Document.java > Document
1 import java.util.*;
2 import java.io.*;
3
4 public class Document {
5     private StringBuffer textBuffer;
6
7     public Document() {
8         textBuffer = new StringBuffer();
9     }
10
11     // Add text from user
12     public void addText() {
13         Scanner in = new Scanner(System.in);
14         System.out.print(s: "Enter text to add: ");
15         String str = in.nextLine();
16         textBuffer.append(str);
17     }
18
19     // Clear all text
20     public void clearText() {
21         textBuffer.setLength(newLength: 0);
22     }
23
24     // Reverse text
25     public void reverseText() {
26         textBuffer.reverse();
27     }
28
29     // Delete portion of text
30     public void deleteText() {
31         Scanner in = new Scanner(System.in);
32         System.out.print(s: "Enter start and end index to delete: ");
33         int a = in.nextInt();
34         int b = in.nextInt();
35
36         if (a >= 0 && a < textBuffer.length() && a < b && b <= textBuffer.length()) {
37             textBuffer.delete(a, b);
38             System.out.println(x: "Text deleted successfully.");
39         } else {
40             System.out.println(x: "Error: Index out of bounds!");
41         }
42     }
}
```

```
Document.java | 2 X Document.java C:\...texteditor DocumentDemo.java TextEditorApp.java text
Document.java > Document > deleteText()
public class Document {
    public void saveToFile(String filename) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            writer.write(textBuffer.toString());
            System.out.println("Text saved successfully to: " + filename);
        } catch (IOException e) {
            System.out.println("Error saving file: " + e.getMessage());
        }
    }

    /**
     * Load text from a file
     */
    public void loadFromFile(String filename) {
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            textBuffer.setLength(newLength: 0); // Clear current content
            String line;
            while ((line = reader.readLine()) != null) {
                textBuffer.append(line).append(str: "\n");
            }
            System.out.println("Text loaded successfully from: " + filename);
        } catch (IOException e) {
            System.out.println("Error loading file: " + e.getMessage());
        }
    }

    // Main method - Demonstration
    Run | Debug
    public static void main(String[] args) {
        Document doc = new Document();
        Scanner sc = new Scanner(System.in);

        System.out.println(x: "=== Simple Text Editor with File Handling ===\n");

        // Add text
        doc.addText();
        doc.displayText();

        // Reverse
        System.out.println(x: "\nReversing text...");
        doc.reverseText();
        doc.displayText();
    }
}
```

cument.java

2

```
80 public static void main(String[] args) {
93     doc.displayText();
94
95     // Save to file
96     System.out.print(s: "\nEnter filename to save (e.g., mytext.txt): ");
97     String saveFile = sc.nextLine();
98     doc.saveToFile(saveFile);
99
100    // Clear and load from file
101    System.out.println(x: "\nClearing current text...");
102    doc.clearText();
103    doc.displayText();
104
105    System.out.println(x: "Loading from file...");
106    doc.loadFromFile(saveFile);
107    doc.displayText();
108
109    // Optional: Delete operation
110    System.out.print(s: "\nDo you want to delete some text? (y/n): ");
111    String choice = sc.nextLine();
112    if (choice.equalsIgnoreCase(anotherString: "y")) {
113        doc.deleteText();
114        doc.displayText();
115    }
116    System.out.print(s: "\nEnter filename to save (e.g., mytext.txt): ");
117    saveFile = sc.nextLine();
118    doc.saveToFile(saveFile);
119
120    System.out.println(x: "Loading from file...");
121    doc.loadFromFile(saveFile);
122    doc.displayText();
123    sc.close();
124    System.out.println(x: "\n=== Program Ended ===");
125 }
126 }
```


OUTPUT

```
Enter filename to save (e.g., mytext.txt): MyText.txt
Text saved successfully to: MyText.txt
```

```
Clearing current text...
Current Content:
Loading from file...
  Text loaded successfully from: MyText.txt
Current Content: lrig a ma I
```

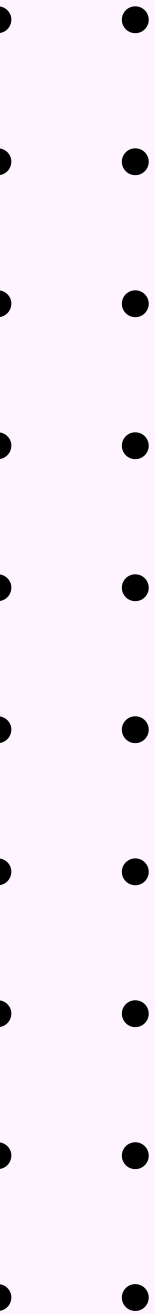
```
Do you want to delete some text? (y/n): Y
Enter start and end index to delete: 3
6
Text deleted successfully.
Current Content: lri ma I
```

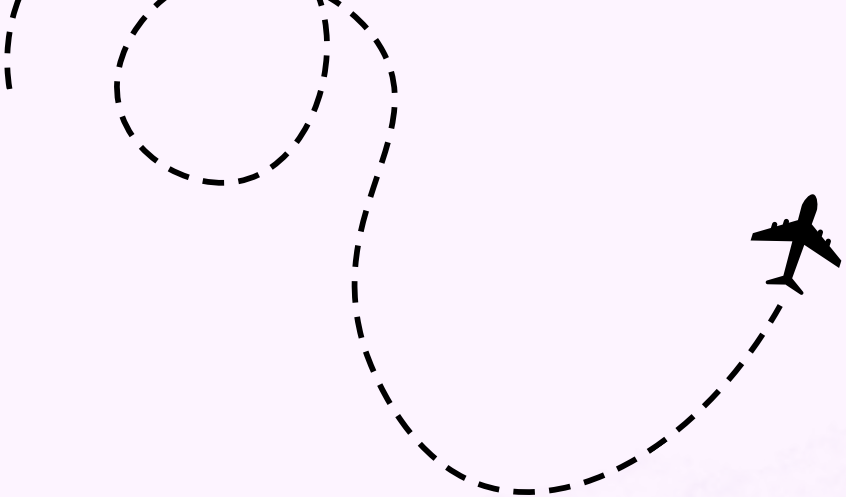
```
Enter filename to save (e.g., mytext.txt): text.txt
Text saved successfully to: text.txt
Loading from file...
  Text loaded successfully from: text.txt
Current Content: lri ma I
```

```
=== Program Ended ===
```

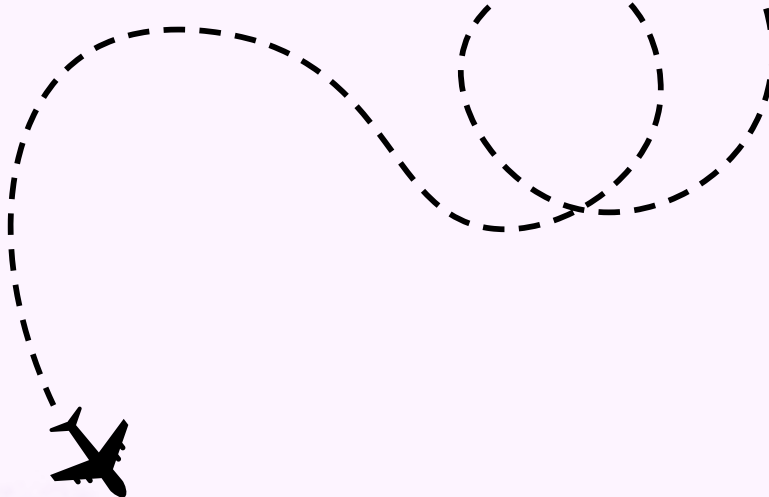

ADVANTAGES OF THE PROJECT

- Simple and easy to understand.
- Demonstrates practical application of OOP.
- Efficient text manipulation using StringBuffer.
- Encourages modular programming.
- Provides a foundation for advanced text editor development.

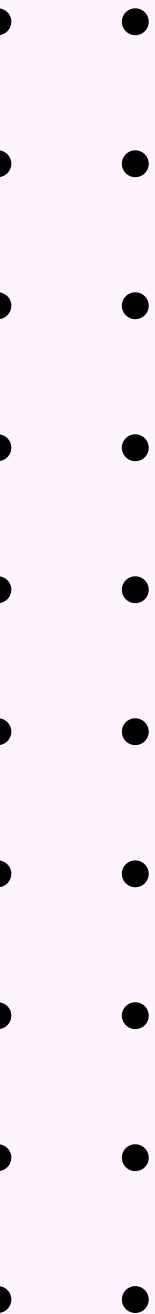




FUTURE SCOPE

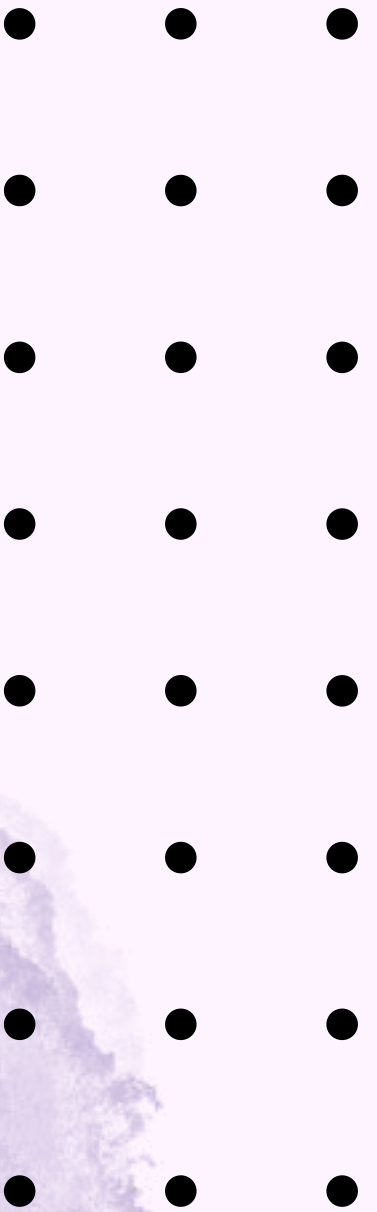


- The project can be enhanced by adding:
- Save and Open File functionality
- Undo and Redo operations
- Find and Replace feature
- Word and Character Count
- Graphical User Interface (GUI) using Java Swing
- Font and Text Formatting Options
- Spell Checking System
- Multi-document support
- These improvements can transform the project into a more advanced text editing application.



CONCLUSION

The Simple Text Editor project successfully demonstrates the implementation of Object-Oriented Programming concepts in Java. By using encapsulation, classes, objects, and StringBuffer operations, the project provides an efficient way to manage and manipulate text. It improves understanding of Java programming and illustrates how real-world software applications are developed using OOP principles.





● ● ● ● ●

THANK YOU

by Apurva Agarwal

